

B ve B+ Ağaçlarının Dünyası



Modern Veritabanları, Dosya Sistemleri ve Büyük Veri Altyapılarının Görünmez Omurgası

Bölüm I: Temeller

Karmaşık veri yapılarını günlük hayattan somut modellerle anlamak.

KÜTÜPHANE ARŞİVİ BENZETMESİ



B-Tree vs B+ Tree Benzetmesi

Bir kütüphane arşivindeki indeks kartı dolaplarını hayal edin:

B-Tree Modelinde: Her çekmecenin içinde hem yönlendirici harf etiketleri (anahtarlar) hem de kitapların fiziksel özetleri (veriler) bir arada bulunur. Dolapta arama yaparken her adımda durup aradığımız veriyi bulup bulmadığımızı kontrol etmemiz gerekir.

B+ Tree Modelinde: İndeks çekmeceleri sadece yönlendirici etiketleri taşır. Asıl kitaplar (veriler) ise sadece en alt raftadır ve birbirine yan yana zincirlenmiştir. Bir kitabı bulduktan sonra bir sonrakine geçmek için tekrar yukarı katlardaki indeks çekmecelerine dönmeniz gerekmez; doğrudan sağa doğru ilerleyebilirsiniz.

B-TREE VE B+ TREE FARKLARI

B-Tree Yapısı

- Veriler hem iç (internal) yönlendirici düğümlerde hem de en alttaki yaprak (leaf) düğümlerde saklanır.
- Tekil aramalarda, aranan eleman üst seviyelerde denk gelirse arama erken bitebilir ve performans $O(1)$ ile $O(\log N)$ arasında değişir.
- Ancak ardışık (aralık) sorgularında ağaç üzerinde sürekli yukarı ve aşağı yönde dolaşmak (In-order traversal) gerektiği için disk okuma maliyeti artar.

B+ Tree Yapısı

- Veriler istisnasız bir şekilde yalnızca en alt seviyedeki yaprak (leaf) düğümlerde depolanır. Üst düğümler sadece rehberlik eder.
- Tüm yaprak düğümleri, birbirine çift yönlü bağlı liste (doubly-linked list) yapısı ile yatay olarak bağlanmıştır.
- Bu yatay zincir sayesinde, aralık sorguları (örneğin "tarih > 2020" araması) yukarıdaki köklere hiç uğramadan, doğrudan yapraklar üzerinde kayarak mükemmel bir hızda yapılır.



Bölüm II: Veritabanları

MySQL, PostgreSQL ve MongoDB gibi veritabanı devlerinin arkasındaki mühendislik kararları.

VERİTABANLARINDA AĞAÇ KULLANIMI

Sistem	Kullanılan Ağaç	Veri Depolama Yapısı	Tercih Nedeni
MySQL (InnoDB)	B+ Tree	"Clustered Index" (Sıkıştırılmış İndeks) ile verileri doğrudan yapraklarda tutar.	Sıkça kullanılan sıralı listeleme (ORDER BY) ve aralık sorgularını (BETWEEN) optimize etmek.
PostgreSQL	B+ Tree Varyantı	Varsayılan indeks türü "nbtree" olup, yüksek oranda optimize edilmiş bir B+ Tree modelidir.	Yüksek eşzamanlılık (concurrency) sağlarken disk blok boyutuyla tam uyum kurarak ACID güvenliğini korumak.
MongoDB	B-Tree	WiredTiger depolama motorunun birincil indekslerinde doğrudan B-Tree yapısı tercih edilir.	İlişkisel olmayan (NoSQL) yapısı gereği tekil belge sorgularında üst düğümlerden veriyi hızlıca dönmek.

Bölüm III: Dosya Sistemleri

İşletim sistemlerinin disk yönetiminden, arama motorlarının devasa indekslerine.

DOSYA SİSTEMLERİ VE BÜYÜK VERİ



Dosya Sistemleri

Windows (NTFS), Linux (ext4, Btrfs) ve macOS (APFS) sistemleri dosya metaverilerini ve izin ağaçlarını B-Tree türevleriyle saklar. Bu sayede klasördeki milyonlarca dosyaya saniyeler içinde erişilir.



Arama Motorları

Arama motorlarının temelini oluşturan "Ters Yüz Edilmiş İndeks" (Inverted Index) yapıları, kelime sözlüklerini ve doküman konumlarını arka planda B-Tree benzeri blok tabanlı yapılarla hızla sorgular.



Büyük Veri

Cassandra, HBase gibi NoSQL sistemleri bellek içi indeks bloklarını B+ Tree prensiplerine göre organize ederek, büyük miktardaki veride bile öngörülebilir okuma/yazma dengesi oluşturur.

NEDEN TERCİH EDİLMEKTEDİR?

%99

Disk Erişim (I/O) Tasarrufu

Donanım Seviyesinde Optimizasyon

Disk Bloklarıyla Kusursuz Uyum: Sabit sürücüler (HDD/SSD) veriyi byte bazında değil, "blok" (genellikle 4KB) halinde okur. B ve B+ Ağaçlarının düğüm büyüklükleri tam olarak bu blok boyutlarına denk gelecek şekilde tasarlanmıştır. Tek bir disk okumasıyla yüzlerce anahtar belleğe yüklenir.

Sığ ve Yayvan Tasarım: İkili arama ağaçları (BST) milyonlarca veri için çok derinleşirken, B+ Ağaçları yüksek dallanma katsayısı (order) sayesinde milyarlarca veride bile 3 veya 4 katman derinliğinde kalır. Bu da aranan veriye en fazla 3-4 disk seek işlemiyle ulaşılmasını sağlar.

GELECEĞE DAİR UZMAN GÖRÜŞLERİ

Bu veri yapıları neden önemlidir?

Bellek (RAM) ve fiziksel diskler arasındaki devasa hız uçurumunu kapatırlar. CPU'lar nanosaniyelerle çalışırken, disk okumaları milisaniyeler sürer. Bu ağaçlar, disk kafasının hareketini ve SSD okuma döngülerini minimumda tutarak yazılım performansının tıkanmasını önler.

Büyük veri çağında neden ihtiyaç duyulmaktadır?

Verinin katlanarak büyüdüğü günümüzde, belleğe sığmayan veri tabanlarını yönetmenin tek yolu akıllı disk I/O yönetimidir. B+ Tree, veri boyutu petabayt seviyesine ulaşsa dahi öngörülebilir ve kararlı bir $O(\log N)$ erişim süresi garantisi sunar.

Gelecekte de kullanılmaya devam edecek midir?

Kesinlikle evet. NVMe SSD'ler ve kalıcı bellek (NVM) teknolojileri gelişse de, verilerin bloklar halinde organize edilme ihtiyacı değişmemektedir. Ağaç yapıları yeni donanım mimarilerine adapte olarak (örneğin donanım önbelleği uyumlu "cache-conscious B-Trees") varlığını korumaya devam edecektir.

IMAGE SOURCES

 Thumbnail https://i.etsystatic.com/21352732/r/il/4e76bf/5417610193/il_fullxfull.5417610193_ksqn.jpg
for Source: www.etsy.com
www.etsy.com